



Universidad
de Valparaíso
CHILE

Manual de Usuario

Transformador XML-JATS

Universidad de Valparaíso

Facultad de Medicina

Autor: Cristian Carreño León

Fecha: 12-04-2026

Versión: 0.7.5

Contacto: cristian.carreno@uv.cl

Documentación General (README)

Transformador XML JATS

> v0.7.5 · Python 3.9+ · Streamlit · JATS 1.3 / 1.4 (ANSI/NISO Z39.96-2024)

Una herramienta avanzada impulsada por Inteligencia Artificial Multi-Agente para convertir documentos de Word (.docx) y PDF (.pdf) al estándar JATS XML validado, diseñada específicamente para el flujo editorial de revistas científicas.

El sistema automatiza el etiquetado semántico completo del artículo, la extracción de metadatos, tablas e imágenes, y la validación contra el estándar NISO JATS Version 1.4 (ANSI/NISO Z39.96-2024), con soporte para múltiples proveedores de IA (Gemini, OpenAI, Anthropic, DeepSeek, Mistral, Groq, Ollama local).

Características Principales

Conversión y Etiquetado

- Soporte Multiformato: Procesa documentos Word (.docx) y PDF (.pdf) directamente.
- Etiquetado Semántico Completo: La IA aplica el esquema JATS completo secciones, párrafos, listas, fórmulas, figuras, tablas y referencias sin alterar el texto original.
- Extracción de Metadatos con IA: Identifica y extrae automáticamente título, autores (con afiliaciones), DOI, ORCID, fechas de recepción, aceptación y publicación.
- Extracción de Tablas: Las tablas se convierten automáticamente a <table-wrap> con <thead>/<tbody> correctamente estructurados, preservando todas las filas y columnas.
- Preservación de Texto (Zero-Edit): El sistema etiqueta el texto original sin modificarlo, respetando el trabajo de los correctores humanos.

Arquitectura Multi-Agente

- Generación Paralela: Permite generar múltiples versiones del XML simultáneamente usando distintos modelos (Gemini, OpenAI, Claude, etc.) y seleccionar el mejor resultado.
- Leaderboard DTD: Panel competitivo que evalúa en tiempo real la precisión estructural de cada XML generado contra el DTD JATS, asignando un porcentaje de exactitud y coronando al modelo ganador.
- Agente Editorial Experto: Agente de revisión con rol de editor experto que asiste didácticamente en la corrección de errores JATS antes de la exportación final.
- Arquitectura LLM-Agnostic: Soporta Google Gemini, OpenAI (GPT-4o), Anthropic (Claude 3.5), DeepSeek, Mistral, Groq y modelos locales offline con Ollama o LM Studio sin modificar el código.

Validación y Calidad

- Validación JATS 1.3 / 1.4: Valida contra los estándares NISO JATS Version 1.3 y 1.4 (ANSI/NISO Z39.96-2024) usando el DTD oficial empaquetado localmente.

- Auditoría de Integridad Semántica (Semantic Check): Valida heurísticamente la densidad de texto real, bloqueando XMLs con placeholders (<!-- ... -->), secciones vacías o contenido insuficiente generados por modelos de baja capacidad o cuota limitada.
- Corrección Automática Asistida por IA: Si la validación falla, un agente experto sugiere y aplica correcciones directamente sobre el XML generado, con revisión humana antes de aceptar.
- Parsing Resiliente: Recuperación automática ante etiquetas ligeramente malformadas (recover=True), permitiendo generar la vista previa HTML incluso con XML imperfecto.

Salida y Exportación

- Conversión a HTML5 Responsivo: Genera HTML autocontenidos con logo incrustado (Base64), modo claro/oscuro, tabla de contenidos interactiva y DOIs clickeables.
- Vista Previa en Nueva Pestaña: La previsualización se abre en el navegador con soporte UTF-8 completo y navegación por anclas internas.
- Descarga Directa: Botones para descargar el XML JATS final y el HTML generado.

Gestión y Seguridad

- Gestión Multi-Proveedor de API Keys: Las claves se guardan localmente en SQLite con ofuscación Base64; nunca se envían a servidores externos.
- Monitoreo de Tokens: Panel estadístico con historial de 30 días, alertas al 80 % y 100 % de la cuota gratuita, y métricas por proveedor.
- Detección de Cuota Agotada: Cuando se agota la cuota gratuita el sistema detecta el error 429 automáticamente y muestra un banner informativo en la barra lateral.
- Metadatos de Revista Persistentes: Título de revista, editorial e ISSN se configuran una sola vez y se inyectan en todas las transformaciones.
- Interfaz Dual:
- Web UI (Streamlit): Interfaz gráfica con arrastrar-y-soltar, revisión de metadatos, chatbot asistente y previsualización en tiempo real.
- CLI (Línea de Comandos): Para automatización, procesamiento por lotes e integración en pipelines editoriales.

Requisitos del Sistema

Requisito	Mínimo	Recomendado
Sistema Operativo	Windows 10, macOS 11, Ubuntu 20.04	macOS 13+ / Ubuntu 22.04
Python	3.9	3.11+
RAM	4 GB	8 GB
Almacenamiento	500 MB	1 GB
Conexión a Internet	Requerida (para proveedores cloud)	

API Key requerida de al menos uno de los proveedores soportados (Gemini, OpenAI, Anthropic, DeepSeek, Mistral, Groq) o una instalación local de [Ollama](https://ollama.com) ejecutándose en localhost:11434.

Instalación

Instalación Estándar

1. Clonar el repositorio:

```
git clone https://github.com/iCristian/XML-JATS.git
cd XML-JATS
```

2. Crear un entorno virtual (recomendado para aislar dependencias):

```
python -m venv .venv
```

Activar:

Windows PowerShell: `.venv\Scripts\Activate.ps1`

Windows CMD: `.venv\Scripts\activate.bat`

macOS/Linux: `source .venv/bin/activate`

3. Instalar dependencias:

```
pip install -r requirements.txt
```

4. Configurar la API Key (elige una opción):

Opción A Desde la interfaz web (recomendado):

Abre la aplicación y navega a " API y Tokens" en el menú lateral. Selecciona tu proveedor de IA (Gemini, OpenAI, Anthropic, etc.), pega tu clave y presiona " Guardar". La clave se almacena en data/config.db (excluido de Git) con ofuscación Base64.

Opción B Variable de entorno:

Google Gemini

```
export GEMINI_API_KEY="Alza..."
```

OpenAI

```
export OPENAI_API_KEY="sk-..."
```


Anthropic

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

> Modelos locales con Ollama: No requieren API Key. Instala [Ollama](https://ollama.com), descarga un modelo (ollama pull llama3) y asegúrate de que el servicio esté corriendo. El sistema lo detecta automáticamente en http://localhost:11434.

> Nota sobre cuota pagada (Gemini): Si tu cuenta de Google AI tiene facturación habilitada en Google Cloud, la misma clave funciona automáticamente en el tier de pago una vez agotada la cuota gratuita. No se necesita ninguna clave adicional.

Uso

Interfaz Web (Recomendado)

```
streamlit run streamlit_app.py
```

o usando el script helper:

```
./run_app.sh
```

Abre tu navegador en http://localhost:8501. El flujo de trabajo es:

1. Configurar tu API Key en "API y Tokens".
2. Cargar un archivo .docx o .pdf en el Transformador.
3. Revisar los metadatos extraídos por la IA (título, autores, DOI, etc.).
4. Generar el XML JATS opcionalmente con múltiples modelos en paralelo.
5. Validar el XML contra el DTD JATS oficial y corregir si es necesario.
6. Descargar el XML validado y/o el HTML para publicación web.

Línea de Comandos (CLI)

Para usuarios avanzados o integración en pipelines editoriales.

Transformación (Word XML JATS):

```
python -m modules.transformer entrada.docx salida.xml
```

Conversión (XML JATS HTML5):

```
python -m modules.xml_html entrada.xml salida.html
```

Verificar modelos disponibles:

```
python check_models.py
```

Estructura del Proyecto

XML-JATS/

streamlit_app.py # Punto de entrada web (Streamlit multi-página)
run_app.sh # Script helper para lanzar la app
requirements.txt # Dependencias Python (pinned)
README.md # Este archivo
RESUME.md # Resumen técnico del proyecto
CONTRIBUTING.md # Guía de contribución

modules/ # Backend independiente de la UI

init.py

transformer.py # Núcleo de conversión DOCX/PDF XML JATS
metadata_processor.py # Extracción de metadatos con IA
correction.py # Corrección asistida por IA (preserva texto original)
prompts.py # Repositorio centralizado de prompts para la IA
xml_html.py # Conversión XML JATS HTML5 responsivo
config_store.py # Persistencia SQLite (API keys, tokens, métricas)
llm_provider.py # Abstracción multi-proveedor (LLMProvider)
convert_images.py # Utilidad de extracción/conversión de imágenes
dtd/ # DTDs JATS empaquetados para validación offline
 JATS-Publishing-1-3-MathML3-DTD/

views/ # Frontend vistas Streamlit

transformador.py # Vista principal: carga, metadatos, generación, validación
configuracion.py # Gestión de API keys, cuotas y datos de revista
manual_usuario.py # Manual de usuario interactivo con pasos y FAQ
documentacion.py # Documentación técnica (README, CONTRIBUTING, RESUME)
creditos.py # Créditos, privacidad y licencias

resources/ # Recursos estáticos

UV_blanco.png # Logo UV (fondo blanco)
UV_color.png # Logo UV (color)
logo.png # Logo principal
logo_transparent.png # Logo transparente
manual_images/ # Capturas de pantalla del manual de usuario

data/ # Datos locales excluidos de Git

config.db # Base de datos SQLite (API keys, tokens, métricas)

Descripción de Módulos Backend

Módulo	Responsabilidad
transformer.py	Orquesta el flujo completo: extrae texto/tablas/imágenes del DOCX
metadata_processor.py	Extrae metadatos (título, autores, DOI, ORCID, afiliaciones, fechas)
correction.py	Agente de corrección que recibe el XML con errores DTD y genera
prompts.py	Repositorio centralizado de todos los prompts maestros. Ningún ot
xml_html.py	Convierte XML JATS a HTML5 responsivo con logo incrustado (Ba
config_store.py	Gestión de persistencia mediante SQLite: guarda/carga API keys (
llm_provider.py	Clase abstracta LLMProvider con implementaciones para Gemini,

Flujo de Datos

Usuario sube DOCX/PDF

transformer.extraer_contenido_estructurado()
python-docx / pdfplumber texto plano + tablas + imágenes
(tablas renderizadas como markdown para el prompt)

metadata_processor.MetadataExtractor
LLMProvider.generate() JSON con título, autores, DOI, fechas

[Revisión humana y chatbot asistente en UI]

transformer.generar_xml_jats() [opcionalmente en paralelo con múltiples modelos]
LLMProvider.generate() XML JATS candidato

Auditoría de Completitud Semántica (verificar_completitud_xml)
Detecta placeholders (<!-- ... -->)
Valida densidad de texto vs. original
Puntúa cada candidato

Leaderboard: selección del XML con mayor puntaje DTD

[Si errores] correction.corregir_xml()
LLMProvider.generate() XML corregido

xml_html.build_html() HTML5 autocontenido con DOIs clickeables

Descarga: XML + HTML

Proveedores de IA Soportados

Proveedor	Modelos por Defecto	Tier Gratuito	Variable de Entorno
Google Gemini	gemini-2.5-flash, gemini-2.5-pro (GPT-4o equivalent)	Sí (120 días)	GEMINI_API_KEY
OpenAI	gpt-4o, gpt-4o-mini, gpt-4-turbo	Pay-as-you-go	OPENAI_API_KEY
Anthropic	claude-3-5-sonnet, claude-3-haiku	Pay-as-you-go	ANTHROPIC_API_KEY
DeepSeek	deepseek-chat, deepseek-coder	Limitado	DEEPSEEK_API_KEY
Mistral	mistral-large, mistral-small	Limitado	MISTRAL_API_KEY
Groq	llama3-70b, mixtral-8x7b	Sí	GROQ_API_KEY
Ollama (local)	llama3, qwen2.5, etc.	Sin límites	(local)
LM Studio (local)	cualquier modelo GGUF	Sin límites	(local)

Cuotas Gemini (Tier Gratuito)

Modelo	Req/día	Tokens/min	Req/min
gemini-2.5-flash	500	250 000	10
gemini-2.5-pro	25	32 000	5
gemini-2.0-flash	1 500	1 000 000	15

> Una vez agotada la cuota gratuita, la misma clave funciona en Pay-as-you-go si la cuenta tiene facturación habilitada en Google Cloud.

Variables de Entorno

Variable	Proveedor	Descripción
GEMINI_API_KEY	Google	API Key de Google Gemini
OPENAI_API_KEY	OpenAI	API Key de OpenAI
ANTHROPIC_API_KEY	Anthropic	API Key de Anthropic
DEEPSEEK_API_KEY	DeepSeek	API Key de DeepSeek
MISTRAL_API_KEY	Mistral	API Key de Mistral
GROQ_API_KEY	Groq	API Key de Groq

Las variables de entorno funcionan como fallback si no hay clave guardada en data/config.db. Si ambas existen, prevalece la guardada en la base de datos.

Solución de Problemas

Error: ModuleNotFoundError

Asegúrate de tener el entorno virtual activado

```
source .venv/bin/activate  
pip install -r requirements.txt
```

Error: SSL: CERTIFICATE_VERIFY_FAILED (macOS)

Instala los certificados de Python

/Applications/Python\ 3.x/Install\ Certificates.command

El sistema también tiene un mecanismo de fallback que ignora errores SSL al descargar el DTD JATS.

Error 429: Cuota Agotada

- Espera al día siguiente (los límites diarios se renuevan a las 00:00 UTC).
- Cambia a un modelo con más cuota (ej. gemini-2.0-flash tiene 1500 RPD).
- Configura otro proveedor (OpenAI, Anthropic, etc.) como alternativa.
- Activa facturación en Google Cloud para usar el tier Pay-as-you-go con la misma clave.

El XML generado contiene placeholders <!-- ... -->

- El sistema detecta esto automáticamente en la Auditoría de Completitud.
- Si igual ocurre, usa el botón " Intentar Solucionar con IA" en la sección de Validación.
- Si persiste, prueba con un modelo de mayor capacidad (ej. gemini-2.5-pro en lugar de flash).

Ollama no conecta

Verificar que el servicio esté activo

ollama list

Si no está corriendo, iniciarlo manualmente

ollama serve

Las imágenes del PDF no se extraen

Los PDFs con imágenes vectoriales o con protección DRM pueden no permitir la extracción. Usa el archivo Word original si está disponible.

Créditos

Desarrollado por Cristian Carreño León\
Escuela de Obstetricia y Puericultura\
Facultad de Medicina\
Universidad de Valparaíso, Chile\
cristian.carreno@uv.cl

Desarrollado para la Universidad de Valparaíso con el objetivo de optimizar los procesos de publicación científica.

Tecnologías Utilizadas

Tecnología	Versión	Uso
Python	3.9+	Lenguaje base
Streamlit	1.51+	Framework de interfaz web
Google Gemini	2.5-flash	LLM principal para etiquetado
lxml	6.0+	Procesamiento y validación XML/HTML
python-docx	1.2+	Extracción de contenido Word
pdfplumber	0.10+	Extracción de texto desde PDF
tenacity	9.1+	Reintentos robustos con backoff exponencial
FPDF2		Generación del manual en PDF
SQLite		Persistencia de API keys y métricas
JATS	1.3 / 1.4	Estándar ANSI/NISO Z39.96-2024

Licencia

Este software es de uso exclusivo para la Universidad de Valparaíso. Todos los derechos reservados.

El código fuente y la documentación contenidos en este repositorio son propiedad intelectual de la Universidad de Valparaíso y su autor. Queda prohibida su reproducción, distribución o modificación sin autorización expresa.

Para consultas sobre licenciamiento o uso, contactar a:
cristian.carreno@uv.cl

Historial de Versiones (Changelog)

v0.7.5 Auditoría Semántica y Robustez de Infraestructura

- Verificación de Completitud Semántica: Nueva capa de validación heurística que detecta omisiones de texto, placeholders de IA e insuficiencia de densidad de datos, garantizando la integridad de la investigación maquetada.
- Auto-vuelo de Ollama (macOS): Implementación de auto-detección y lanzamiento del servicio local Ollama. Si el servicio no responde, el sistema intenta levantarlo automáticamente en entornos macOS para garantizar disponibilidad inmediata.
- Etiquetado de Origen [LOCAL/CLOUD]: Identificación clara de modelos en el selector, diferenciando entre inferencia local (Ollama) y proveedores cloud, con monitoreo de estado en tiempo real.
- Refuerzo de Prompt "Zero-Placeholder": Instrucciones maestras rediseñadas para prohibir estrictamente el uso de comentarios XML como marcadores de posición, forzando la transcripción íntegra del cuerpo y referencias.

v0.7.0 Arquitectura Multi-Agente y Multi-Model (Vendor-Agnostic)

- Generación Paralela Multi-Agente: Permite generar múltiples versiones de XML simultáneamente utilizando modelos distintos (Gemini, OpenAI, Anthropic, etc.), evaluando su rendimiento en tiempo real.
- Leaderboard y Puntuación DTD: Implementación de un panel competitivo que evalúa automáticamente la precisión estructural de cada XML generado contra las estrictas reglas del DTD JATS. Genera un porcentaje de exactitud y corona a un modelo ganador.
- Validador Editorial AI (Agente Experto): Incorpora un agente de revisión con rol de Editor Experto que asiste con explicaciones didácticas para subsanar los posibles fallos JATS del XML final, de forma detallada antes de su exportación.
- Soporte Multi-Proveedor (LLM-Agnostic): El Transformador XML abandona el confinamiento a Gemini e introduce una robusta arquitectura con soporte nativo para OpenAI, Anthropic (Claude), DeepSeek, Mistral, Groq e inferencia local offline apoyada en Ollama (Llama3, Qwen, etc).
- Abstracción Multi-Proveedor: Un nuevo diseño estructural (modules/llm_provider.py) introduce la clase abstracta LLMProvider permitiendo la fácil y limpia integración de nuevos ecosistemas y API providers en el futuro.
- Selectificador y Dashboard Híbrido: El menú de *Configuración* fue reconstruido para permitir el ingreso de la API Key particular a cada servicio con enmascaramiento Base64, además de proveer atajos URL para expedir las Key gratuitas de todos los competidores en el mercado.

- Cuotas Multi-Cuenta y Fallbacks: Rebalanceo interno del monitoreo de peticiones e historial contable, el token-meter reportará fidedignamente según el proveedor utilizado.

v0.68 Metadatos de Revista Persistentes, Selectores DTD y Corrección SSL

- Metadatos de Revista Persistentes: Nueva sección en configuración para guardar datos estructurales de la publicación (título de la revista, editorial, ISSN), los cuales se inyectan automáticamente en cada nuevo flujo ahorrando escritura manual repetitiva.
- Selector de Versión JATS Dinámico: El usuario puede escoger la versión objetivo del estándar (1.3 o 1.4) en la misma página de configuración. El transformador inyectará condicionalmente la declaración `<!DOCTYPE>` y validará contra el DTD JATS oficial preciso usando rutas dependientes del contexto.
- Validación SSL Opt-out Local: Se resolvió un bug prevalente en macOS (SSL: CERTIFICATE_VERIFY_FAILED) ignorando programáticamente fallos SSL intermitentes al hacer el fetch oficial del DTD JATS a NLM. Esto asegura que la herramienta arranque exitosamente en cualquier estado de certificados Python.
- UI Responsiva y Textos Justificados: Mejorada radicalmente la integración HTML CSS para IFrames como OJS (width: 100%) y se aseguró uniformidad con justificación de texto automática en artículos.

v0.67 Unificación a Clave API Única y Detección de Cuota

- Una sola API Key: Eliminado el concepto de "Key Pro" separada. El sistema ahora usa una única clave de API. Google gestiona internamente el paso de tier gratuito a tier de pago cuando la cuenta tiene facturación habilitada en Google Cloud.
- Detección de Cuota Agotada: Cuando transformer.py o correction.py reciben un error 429 tras reintentos, retornan `{'quota_exceeded': True}` en lugar de intentar cambiar de clave. La UI muestra automáticamente un banner Cuota gratuita agotada usando tier de pago en la barra lateral.
- Tabla de Cuotas Simplificada: La sección "Configuración" reemplaza las columnas "Pro" de la tabla por una única columna "Tras cuota libre: Pay-as-you-go". Se añade un panel informativo explicando el modelo de una sola clave con enlace a [Google AI pricing](https://ai.google.dev/pricing).
- Funciones Obsoletas Marcadas: `save_api_key_pro`, `load_api_key_pro` y `delete_api_key_pro` en `config_store.py` emiten `DeprecationWarning` si se llaman. Se mantienen solo para compatibilidad con bases de datos antiguas.
- Backend Limpio: transformer.py y correction.py ya no aceptan el parámetro `api_key_pro` en sus firmas. Args residuales se absorben con `_kwargs`.

v0.66 Interfaz Minimalista, Validación de API y Configuración Dedicada

- Página de Configuración Exclusiva: Nueva pestaña "Configuración" dedicada exclusivamente a visualizar y gestionar las API Keys y el modelo IA seleccionado, liberando espacio en el menú principal.
- Validación Robusta de API Keys: Se implementó una verificación exhaustiva de la API Key en el backend, la cual ahora detecta placeholders o claves inválidas antes de enviar peticiones a la API, evitando errores confusos y cierres inesperados.

- Interfaz del Menú Lateral Minimalista: Rediseño del panel lateral en el transformador principal. El estado de la API Key ahora se muestra de forma compacta (mini indicador) bajo el seleccionador de modelo. Las instrucciones pasaron de ocupar gran espacio a mostrarse en un práctico "popover" (ventana flotante).
- Recuperación Automática de XML Crítico (Resilient Parsing): El parser XML ahora cuenta con capacidad de recuperación ante etiquetas malformadas menores (`recover=True`), lo cual permite generar un HTML de vista previa incluso si la IA cometió ligeros errores estructurales en el etiquetado JATS.
- Mejores Mensajes de Error: Si el XML está demasiado dañado para generar HTML seguro, la aplicación ahora detecta el fallo y sugiere amablemente al usuario utilizar la pestaña de "Validación y Corrección" asistida por IA para solucionarlo automáticamente.

v0.65 Auditoría Exhaustiva y Corrección de Bugs Críticos

- 15+ Bugs Críticos Corregidos: Auditoría completa del código fuente de todos los módulos con corrección de errores funcionales que impedían el flujo correcto.
- Extracción de Tablas Mejorada: Instrucciones detalladas en el prompt para generar `<table-wrap>` con `<table>`, `<thead>`, `<tbody>`, `<tr>`, `<th>`, `<td>` completos. Las tablas ya no se omiten ni se generan vacías.
- DOI y Fechas Obligatorios: El DOI y la fecha de publicación son ahora campos obligatorios en la validación de metadatos. Instrucciones reforzadas para que el modelo siempre incluya `<pub-date>` y `<article-id pub-id-type="doi">`.
- Secciones con Títulos: Instrucciones mejoradas para que cada sección del artículo (Introducción, Métodos, Resultados, etc.) se mapee correctamente a `<sec><title>`.
- Token Limit: `max_output_tokens` aumentado de 8192 a 65536 los artículos extensos ya no se truncan.
- Corrección de Modelo: Todos los módulos ahora usan gemini-2.5-flash de forma consistente.
- Parsing de Metadatos Robusto: Nuevo método `_normalize_metadata()` que corrige variaciones en las claves del JSON devuelto por la IA. Extracción de JSON mejorada para manejar respuestas con prefijos no-JSON.
- Corrección de Chatbot: El flujo de corrección por chatbot ahora detecta correctamente XML directo (sin backticks) en las respuestas de la IA, resolviendo el bug donde las correcciones nunca se aplicaban.
- Renderización de Referencias HTML: `xml_html.py` ahora maneja correctamente elementos JATS de bibliografía (`<name>`, `<surname>`, `<given-names>`, `<person-group>`, `<source>`, `<pub-id>`, etc.) con DOIs clickeables y fuentes en itálica.
- Eliminación de Código Duplicado: Removida la definición duplicada de `validar_jats_xml` y la llamada a `construir_prompt_avanzado` (función inexistente).

v0.64 Refactorización del Backend y Desacoplamiento de UI

- Arquitectura Desacoplada: Eliminada la dependencia de streamlit (`st.session_state`) en los módulos del backend (`metadata_processor.py`, `correction.py`, `transformer.py`). Ahora toda la configuración (API keys, versión de modelo) se pasa explícitamente, permitiendo ejecución pura por CLI o testing automatizado.
- Centralización de Prompts: Creación del nuevo módulo `modules/prompts.py` que almacena todas las plantillas de instrucciones complejas para la IA, limpiando el código funcional y facilitando la afinación del comportamiento del modelo.

- Reintentos Robustos con Tenacity: El bucle manual de reintentos en las llamadas a la API de Google Gemini ha sido reemplazado por la librería estándar tenacity. Ahora el sistema maneja automáticamente los errores de cuota (HTTP 429) usando retardos exponenciales, mientras preserva el fallo rápido para errores irre recuperables como bloqueos por *Recitation*/Copyright.

v0.63 API Key Persistente y Monitoreo de Tokens

- Almacenamiento Persistente de API Key: La clave de Gemini se guarda de forma segura en una base de datos SQLite local (data/config.db).
- Seguridad de la Key: Una vez guardada, la clave desaparece de la interfaz. Solo se muestra una versión enmascarada (AlzaxY4Z). Botones para cambiar o borrar la key en cualquier momento.
- Panel de Uso de Tokens: Nuevo panel expandible en la barra lateral que muestra:
- Requests usadas hoy vs. límite diario (RPD) con barra de progreso.
- Tokens consumidos hoy.
- Límites por minuto (TPM y RPM) del modelo seleccionado.
- Historial acumulado total.
- Advertencias de Límite: Alertas automáticas al alcanzar el 80% y 100% del límite diario del tier gratuito.
- Registro Automático: Cada operación con IA (generación, corrección, chatbot) registra automáticamente los tokens consumidos.
- Nuevo Módulo config_store.py: Gestión centralizada de configuración y métricas mediante SQLite.

v0.62 Mejoras de HTML, Manejo de Recitación y Deduplicación de Referencias

- Deduplicación de Referencias: Corregido el problema donde los números de referencias bibliográficas aparecían triplicados en el HTML (por , <label> y texto de la cita). Nueva función `_strip_leading_ref_number()` limpia automáticamente los números duplicados.
- Vista Previa en Nueva Pestaña: La previsualización HTML ahora se abre en una pestaña nueva del navegador en lugar de un iframe incrustado, resolviendo problemas con enlaces y estilos.
- Corrección UTF-8: Implementación correcta de decodificación UTF-8 en la vista previa (TextDecoder en lugar de atob), solucionando caracteres acentuados corruptos (ej. "crÃticas" "críticas").
- Navegación por Anclas: Script de interceptación de clicks en enlaces #ancla usando `scrollIntoView()`, corrigiendo la navegación interna en páginas abiertas con `document.write()`.
- Preservación de Texto (Prompt): Prompt de generación JATS completamente reescrito para prohibir modificaciones al texto original la IA solo etiqueta, nunca reescribe.
- Manejo de Recitación: Manejo robusto de errores `finish_reason=4` (recitación) de la API de Gemini con reintentos automáticos y temperatura progresiva.
- Filtros de Seguridad Desactivados: Todos los filtros de seguridad de Gemini configurados en BLOCK_NONE (apropiado para contenido académico publicado).
- Instrucciones de Referencia en Prompt: Las instrucciones JATS ahora especifican explícitamente que el número de referencia va SOLO en <label>, no en <element-citation>.

v0.6 Actualización JATS 1.4 + Metadatos

- Estándar JATS 1.4: Actualización completa del motor de validación y prompts al estándar ANSI/NISO Z39.96-2024.
- Soporte PDF: Ahora es posible cargar archivos .pdf para extracción de texto y metadatos.
- Módulo de Metadatos: Nueva lógica de extracción y validación de metadatos (título, autores, DOI, fecha).
- UI de Revisión: Nueva sección en la interfaz para editar metadatos y chatbot asistente para completar información faltante.

v0.5 (Beta)

- Mejoras de UI: Navegación por pestañas con botones de avance automático.
- Corrección Interactiva: Nuevo chatbot que permite dialogar con la IA para resolver errores de validación. Implementación de una "Caja de Sugerencias" para revisar y aplicar cambios al XML de forma segura.
- Logo Embedded: El logo de la revista ahora se incrusta como Base64 en el HTML generado, eliminando la dependencia de carpetas locales.
- Optimización: Eliminación de animaciones intrusivas y mejora en la legibilidad del chat en modo oscuro.

v0.1 v0.4 (Alpha)

- Inicio del proyecto.
- Configuración de Gemini CLI.
- Extracción básica de DOCX.
- Primera implementación de transformación a XML JATS.

Guía de Contribución

Guía de Contribución

¡Gracias por tu interés en mejorar el Transformador XML JATS! Este documento describe las normas y mejores prácticas para contribuir al código.

Tabla de Contenidos

- [Configuración del Entorno de Desarrollo](#configuración-del-entorno-de-desarrollo)
- [Arquitectura del Proyecto](#arquitectura-del-proyecto)
- [Principios de Diseño](#principios-de-diseño)
- [Estilo de Código y Normas](#estilo-de-código-y-normas)
- [Cómo Añadir un Nuevo Proveedor de IA](#cómo-añadir-un-nuevo-proveedor-de-ia)
- [Flujo de Trabajo y Pull Requests](#flujo-de-trabajo-y-pull-requests)
- [Validación Local](#validación-local)
- [Errores Conocidos y Mitigaciones](#errores-conocidos-y-mitigaciones)
- [Política de Seguridad](#política-de-seguridad)
- [Contacto](#contacto)

Configuración del Entorno de Desarrollo

1. Clonar el repositorio:

```
git clone https://github.com/iCristian/XML-JATS.git
cd XML-JATS
```

2. Crear un entorno virtual (Python 3.9+ recomendado):

```
python -m venv .venv
source .venv/bin/activate # En Windows: .venv\Scripts\activate
```

3. Instalar dependencias:

```
pip install -r requirements.txt
```

4. Configurar la API Key:

Ingresa tu clave desde la pestaña " API y Tokens" en la interfaz web, o exporta la variable de entorno antes de ejecutar:

```
export GEMINI_API_KEY="tu_clave_aqui"
```

```
streamlit run streamlit_app.py
```

5. Verificar la instalación:

Verificar sintaxis de todos los módulos

```
python -m py_compile modules/transformer.py
python -m py_compile modules/llm_provider.py
python -m py_compile modules/metadata_processor.py
```

Verificar modelos disponibles

```
python check_models.py
```

Arquitectura del Proyecto

El proyecto sigue una arquitectura desacoplada donde el backend es independiente de la interfaz gráfica:

```
streamlit_app.py      # Punto de entrada web
views/
  transformador.py    # UI principal (Streamlit)
  configuracion.py    # Configuración de API keys y tokens
  manual_usuario.py   # Manual de usuario interactivo
  documentacion.py    # Documentación técnica (README, CONTRIBUTING)
  creditos.py         # Créditos y licencias
modules/
  transformer.py      # Núcleo de conversión DOCX/PDF XML JATS
  prompts.py         # Plantillas de prompts centralizadas
  metadata_processor.py # Extracción de metadatos con IA
  correction.py       # Corrección asistida por IA
  xml_html.py        # Conversión XML JATS HTML5
  config_store.py     # Persistencia (SQLite) API keys y métricas
  llm_provider.py     # Abstracción Multi-Proveedor y Multi-Agente
```

Principios de Diseño

Cualquier contribución debe respetar estrictamente los siguientes principios:

1. Backend sin dependencia de UI

transformer.py, metadata_processor.py, correction.py, llm_provider.py y prompts.py NO deben importar ni depender de streamlit o st.session_state. La configuración se pasa como parámetros explícitos y se persiste usando config_store.py. Este desacoplamiento es esencial para permitir el uso por CLI y testing automatizado.

2. Arquitectura LLM-Agnostic

El proyecto utiliza la clase abstracta LLMProvider en llm_provider.py para comunicarse con cualquier proveedor de IA. No introduzcas llamadas directas a google.generativeai, openai.OpenAI() u otras APIs fuera de llm_provider.py. Toda llamada a la IA debe pasar por LLMProvider.generate().

3. Prompts Centralizados

Todas las instrucciones para la IA están exclusivamente en prompts.py. Nunca hardcodees prompts en transformer.py, correction.py ni en ninguna vista. Si necesitas un nuevo prompt, agrégalo como función o constante en prompts.py.

4. Modelo y Versión Consistentes

El modelo por defecto es gemini-2.5-flash en toda la aplicación. Si añades un nuevo punto de llamada a la IA, respeta el modelo activo configurado por el usuario (no hardcodees un modelo diferente).

5. Una sola API Key por proveedor

No añadas parámetros api_key_pro ni segundas claves. El sistema ya gestiona el paso entre tier gratuito y pago automáticamente. Cuando la cuota gratuita se agota, transformer.py y correction.py retornan {'quota_exceeded': True} y la UI muestra el banner correspondiente.

6. Señalización de Cuota (No excepciones)

Si una función hace llamadas a la IA y puede recibir error 429, debe retornar un diccionario que incluya 'quota_exceeded': True (no lanzar excepción), para que la UI detecte el estado y actualice el banner lateral. Usa tenacity con stop_after_attempt(3) y wait_exponential() para los reintentos.

7. Política de Integridad Semántica (Zero-Placeholder)

Queda estrictamente prohibido permitir que la IA entregue XMLs con marcadores de posición (<!-- Contenido de... -->). Cualquier cambio en los prompts o el transformador debe reforzar la transcripción íntegra del texto original. La función verificar_completitud_xml() en transformer.py es la última línea de defensa no la elimines ni la debilites.

Estilo de Código y Normas

1. Convenciones Generales

- Seguimos PEP 8.
- Longitud de línea máxima sugerida: 100 caracteres.
- Usa snake_case para funciones y variables, y PascalCase para clases.
- No uses abreviaciones crípticas en nombres de variables.

2. Type Hints (Tipado Estático)

Todas las funciones y métodos deben tener anotaciones de tipo completas para argumentos y valores de retorno.

Correcto:

```
def procesar_texto(entrada: str, opciones: Dict[str, bool]) -> List[str]:  
...
```

Incorrecto:

```
def procesar_texto(entrada, opciones):  
...
```

3. Docstrings (Estilo Google)

Todas las funciones, clases y módulos públicos deben tener docstrings siguiendo el Google Python Style Guide.

```
def funcion_ejemplo(arg1: int, arg2: str) -> bool:  
"""Realiza una operación de ejemplo.
```

Explica brevemente qué hace la función. Puede tener múltiples líneas si es complejo.

Args:

arg1 (int): El primer argumento, que representa X.
arg2 (str): El segundo argumento, que representa Y.

Returns:

bool: True si la operación fue exitosa, False en caso contrario.

Raises:

ValueError: Si arg1 es negativo.
"""

...

4. Manejo de Respuestas de IA

Al procesar respuestas de la IA, sigue estas reglas:

- Parsing de JSON: Siempre busca el primer { y último } para extraer el JSON, ya que el modelo puede anteponer texto libre.
- Parsing de XML: Verifica si la respuesta comienza con < (XML directo) o contiene bloques `<xml>` (markdown). `parse_model_response` en `llm_provider.py` ya limpia los backticks.
- Normalización: Usa `_normalize_metadata()` en `metadata_processor.py` para corregir variaciones en claves del JSON.
- Cuota agotada: Detecta el error 429 con tenacity; si se agotan los reintentos, retorna `{'error': ..., 'quota_exceeded': True}`.
- Auditoría de Completitud: Al modificar el flujo de generación, invoca `verificar_completitud_xml()` para asegurar que el output sea semánticamente denso.

5. Prompts para la IA

Al modificar o añadir prompts en `prompts.py`:

- Incluye instrucciones explícitas para DOI, fechas y secciones con títulos.
- Para tablas, incluye la estructura XML completa como ejemplo en el prompt.
- Usa negritas/mayúsculas para reglas CRÍTICAS que el modelo suele ignorar.
- Siempre incluye recordatorios sobre cierre correcto de etiquetas XML.
- Testea el prompt con al menos un artículo real antes de hacer el PR.

Cómo Añadir un Nuevo Proveedor de IA

1. Crear la clase en `llm_provider.py`:

```
class MiProveedorProvider(LLMProvider):
def generate(self, prompt: str, model: str, api_key: str,
max_tokens: int = 8192) -> str:
```

Implementar llamada a la API del nuevo proveedor

...

```
def get_default_models(self) -> List[str]:
return ["mi-modelo-1", "mi-modelo-2"]

def get_api_key_url(self) -> str:
```



```
return "https://mi-proveedor.com/api-keys"
```

```
def get_api_key_env_var(self) -> str:  
    return "MI_PROVEEDOR_API_KEY"
```

2. Registrar el proveedor en PROVIDER_REGISTRY dentro de llm_provider.py:

```
PROVIDER_REGISTRY["mi_proveedor"] = MiProveedorProvider()
```

3. Añadir los límites de cuota en config_store.py dentro de FREE_TIER_LIMITS (si tiene tier gratuito).
4. Agregar la documentación del proveedor en la sección correspondiente de configuracion.py.

Flujo de Trabajo y Pull Requests

1. Reportar Problemas: Abre un Issue antes de comenzar trabajos grandes. Describe el problema con claridad y, si es posible, incluye un ejemplo de documento que falla.

2. Ramas (Branches): Crea ramas descriptivas desde main:

```
git checkout -b feature/nuevo-proveedor-mistral  
git checkout -b fix/error-parseo-metadatos  
git checkout -b docs/mejorar-manual-usuario
```

3. Commits: Usa mensajes descriptivos en español o inglés:

```
git commit -m "feat: añadir soporte para proveedor Mistral"  
git commit -m "fix: corregir parsing de fechas en metadata_processor"  
git commit -m "docs: expandir manual de usuario con pasos de corrección"
```

4. Pull Requests:

- Describe claramente los cambios realizados y el problema que resuelven.
- Asegúrate de que el código no tenga errores de sintaxis:

```
python -m py_compile modules/transformer.py
```

- Adjunta capturas de pantalla si cambiaste la interfaz gráfica.
- Referencia el Issue relacionado con Closes #123.

Validación Local

Antes de hacer un PR, ejecuta estas validaciones mínimas:

1. Verificar sintaxis de todos los módulos Python

```
for f in modules/*.py views/*.py streamlit_app.py; do
```

```
python -m py_compile "$f" && echo "OK: $f"
done
```

2. Verificar importaciones

```
python -c "from modules import transformer, metadata_processor, correction, llm_provider, config_store,
prompts, xml_html; print('Imports OK')"
```

3. Lanzar la app y verificar manualmente el flujo básico

```
streamlit run streamlit_app.py
```

Errores Conocidos y Mitigaciones

Problema	Causa	Mitigación
La IA antepone texto antes del JSON de Output	Contaminamiento variable del modelo	Extracción por {...} en metadata_processor.py
La IA no cierra etiquetas XML correctamente	Límite de contexto / temperatura	Instrucciones reforzadas en prompts.py + sanitización
max_output_tokens insuficiente para artículos largos y extenso	Artículos largos y extenso	Configurado a 65 536 tokens; considerar chunking
Backticks en la respuesta XML	Modelo responde en markdown	Detección dual de XML (directo o con backticks)
Cuota gratuita agotada (HTTP 429 persis)	Límite diario alcanzado	transformer.py retorna quota_exceeded: True;
SSL: CERTIFICATE_VERIFY_FAILED en Windows	Certificados Python no instalados	Ignorado programáticamente con fallback; instalar certificados
Tablas anidadas no se convierten correctamente	Clasificación del formato JATS	Aplanar la tabla manualmente en el Word original
Placeholders <!-- Contenido... --> en el XML	Modelo de baja capacidad o cuota baja	verificar_completitud_xml() detecta y rechaza;

Política de Seguridad

- No commitas API keys ni secretos en el código fuente. Las claves se almacenan en data/config.db (excluido de Git mediante .gitignore).
- No expongas data/config.db en PRs o capturas de pantalla.
- Si descubres una vulnerabilidad de seguridad, repórtala directamente por correo a cristian.carreno@uv.cl antes de abrir un Issue público.
- Al añadir nuevas dependencias, verifica que no tengan vulnerabilidades conocidas (CVE) en la versión a utilizar.

Contacto

Para dudas técnicas o de arquitectura, contacta a:

Cristian Carreño León cristian.carreno@uv.cl

Universidad de Valparaíso, Facultad de Medicina.

Manual de Usuario

El Transformador XML JATS es una herramienta especializada diseñada para optimizar y automatizar el flujo de trabajo editorial de la Universidad de Valparaíso.

Esta solución convierte manuscritos en formato Microsoft Word (.docx) o PDF al estándar JATS XML (Journal Article Tag Suite), asegurando el cumplimiento con los requisitos de indexación y preservación digital de revistas científicas (OJS, PubMed Central, SciELO, Latindex, etc.).

Características principales:

- Extracción automática de metadatos con IA (título, autores, DOI, fechas, ORCID, afiliaciones).
- Arquitectura Multi-Agente: genera múltiples versiones del XML con distintos modelos de IA y selecciona la mejor.
- Validación automática contra el DTD oficial JATS 1.3 / 1.4 (ANSI/NISO Z39.96-2024).
- Corrección asistida por IA cuando se detectan errores de estructura XML.
- Generación de HTML5 responsivo con DOIs clickeables y tabla de contenidos interactiva.
- Soporte para múltiples proveedores de IA: Google Gemini, OpenAI, Anthropic, DeepSeek, Mistral, Groq y modelos locales (Ollama, LM Studio).

Flujo de Trabajo

Paso 1 Configurar la Clave de API

Antes de usar el Transformador, necesitas configurar al menos una clave de API de un proveedor de IA.

Cómo configurar tu API Key:

1. En el menú lateral izquierdo, haz clic en "API y Tokens".
2. En la sección "Proveedor de IA Activo", selecciona tu proveedor preferido (se recomienda Google Gemini para empezar, ya que ofrece tier gratuito generoso).
3. Haz clic en la pestaña de tu proveedor dentro de "Claves de API".
4. Pega tu API Key en el campo de texto y presiona "Guardar".
5. Una vez guardada, la clave desaparece de la interfaz y se muestra enmascarada (ej. AlzaxY4Z).

¿Dónde obtener una API Key gratuita?

- Google Gemini: aistudio.google.com/apikey 500 requests/día gratuitas con gemini-2.5-flash.
- OpenAI: platform.openai.com/api-keys pay-as-you-go.
- Groq: console.groq.com/keys tier gratuito disponible.

- Ollama (local): No requiere clave. Instala desde ollama.com y ejecuta localmente sin costo.

> Seguridad: Tu clave se almacena localmente en data/config.db con ofuscación Base64. Nunca se envía a servidores externos distintos al proveedor de IA seleccionado.

Imagen correspondiente al Paso 1



Figura 1: Panel de Configuración de API Keys con proveedor activo y estado de cada clave configurada.

Paso 2 Configurar los Datos de la Revista

Para evitar reescribir los mismos datos en cada artículo, configura los metadatos persistentes de tu revista.

Cómo configurar los datos de la revista:

1. En " API y Tokens", baja hasta la sección " Datos por Defecto y Estándar JATS".
2. Rellena los campos:
 - Título de la Revista: nombre oficial completo (ej. Revista Médica de Chile).
 - Editorial (Publisher): institución que publica la revista (ej. Universidad de Valparaíso).
 - ISSN: número ISSN de la revista (ej. 0034-9887).
3. Selecciona la Versión JATS a generar:
 - JATS 1.4 (recomendado): soporte mejorado para accesibilidad y matemáticas.
 - JATS 1.3: usa solo si tu sistema editorial (OJS legacy) lo requiere específicamente.
4. Presiona " Guardar Datos".

Estos datos se inyectarán automáticamente en el XML de cada artículo que proceses, sin necesidad de ingresarlos manualmente cada vez.

Imagen correspondiente al Paso 2

Aplicación

Transformador

Configuración

API y Tokens

Información

Manual de Usuario

Documentación

Créditos y Licencias

Datos de la Revista

Título de la Revista:

Nombre de la Editorial (Publisher):

ISSN de la Revista:

Guardar Datos

Estándar XML JATS

JATS 1.4 añade mejoras en accesibilidad, matemáticas y afiliaciones estructuradas. Selecciona 1.3 solo si tu publicador (ej. OJS legacy) lo restringe.

Versión a generar:

1.4

Cuotas de Gemini por Modelo y Cuenta

Tier Gratuito: Las claves de Google AI Studio operan en capa gratuita con límites diarios y por minuto. Mantenerte bajo este umbral garantiza coste cero. **Atención:** Si vinculaste tu clave a un proyecto de Google Cloud con facturación activa, al sobrepasar estos límites pasarás automáticamente al modelo Pay-as-you-go incurriendo en cargos. Sugerimos `gemini-2.5-flash` para mantener amplio margen de capa libre.

Modelo	Req/día (Free)	Tokens/min (Free)	Req/min (Free)	Tras cuota libre
gemini-2.5-flash	500	250,000	10	Pay-as-you-go*
gemini-2.5-flash-lite	500	250,000	10	Pay-as-you-go*
gemini-2.5-pro	25	250,000	5	Pay-as-you-go*

Figura 2: Configuración de los datos permanentes de la revista y selección del estándar JATS a generar.

Paso 3 Monitorear tu Cuota de Tokens

El panel de consumo te permite controlar el uso de tu cuota gratuita y evitar interrupciones.

Cómo usar el panel de consumo:

1. En "API y Tokens", desplázate hasta la sección "Consumo de Tokens".
2. Verás 4 métricas clave:
 - Requests hoy: cuántas solicitudes has hecho hoy vs. el límite diario del tier gratuito.
 - Tokens hoy: cantidad total de tokens procesados en el día.
 - Total histórico: acumulado de todos los tokens desde que empezaste a usar la herramienta.
 - Operaciones totales: número total de transformaciones y correcciones realizadas.
3. La barra de progreso muestra visualmente el porcentaje de cuota consumida:
 - Verde: menos del 80% consumido estás bien.
 - Amarillo: entre 80% y 100% precaución, estás cerca del límite.
 - Rojo: cuota agotada las solicitudes fallarán hasta que se renueve (00:00 UTC).
4. Los gráficos de uso diario muestran el historial de los últimos 30 días.

Tip: Si tu cuota se agota con frecuencia, considera cambiar al modelo gemini-2.0-flash (1500 requests/día gratuitas) o activar la facturación en Google Cloud para uso ilimitado Pay-as-you-go.

Imagen correspondiente al Paso 3



Figura 3: Panel de consumo de tokens con métricas diarias, históricas y gráfico de uso de los últimos 30 días.

Paso 4 Preparar el Documento Word

La calidad del XML generado depende directamente de la estructura del documento fuente. Sigue estas recomendaciones antes de cargar tu archivo.

Buenas prácticas para el documento Word (.docx):

Estructura recomendada:

- Usa estilos de Word (Título 1, Título 2, etc.) para marcar secciones el sistema los detecta automáticamente.
- El título del artículo debe estar al inicio, en una línea sola, con letra grande o estilo "Título".
- Los nombres de autores deben aparecer bajo el título, idealmente con ORCID y afiliación.
- El DOI debe estar visible (ej. DOI: 10.1234/revista.2024.001).
- Las secciones estándar (Introducción, Métodos, Resultados, Discusión, Conclusiones) deben tener título explícito.
- Las referencias deben estar al final, numeradas o en formato APA/Vancouver.

Tablas:

- Usa las herramientas de tabla de Word (no tablas dibujadas manualmente).
- La primera fila debe ser el encabezado de la tabla.
- Evita celdas combinadas complejas que puedan dificultar la extracción.

Qué evitar:

- Tablas anidadas (tablas dentro de tablas).
- Texto en cuadros de texto flotantes el sistema puede no extraerlos.
- Imágenes con texto incrustado (prefiere texto real en el documento).
- Ecuaciones como imágenes las ecuaciones de Word (editor de ecuaciones) se extraen mejor.

> Soporte PDF: También puedes subir .pdf, pero el formato .docx es siempre preferible ya que permite una extracción más completa y estructurada.

Paso 5 Cargar el Archivo

Una vez que tu documento está listo, es momento de cargarlo en el Transformador.

Cómo cargar el archivo:

1. En el menú lateral, navega a " Transformador".
2. Verás el área de carga de archivos con el texto "Arrastra y suelta tu archivo aquí" o el botón "Browse files".
3. Selecciona o arrastra tu archivo .docx o .pdf.
4. El sistema comenzará automáticamente a:
 - Extraer el texto completo del documento.
 - Detectar y extraer las tablas.
 - Identificar imágenes (en el caso de DOCX).
 - Invocar la IA para extraer los metadatos del artículo.
5. Mientras se procesa, verás un indicador de progreso.

Formatos soportados:

- .docx Microsoft Word (recomendado)
- .pdf Documento PDF

> El tiempo de procesamiento varía según el tamaño del artículo: un artículo típico de 8-12 páginas tarda entre 15 y 45 segundos dependiendo del proveedor de IA y la conexión a internet.

Imagen correspondiente al Paso 5

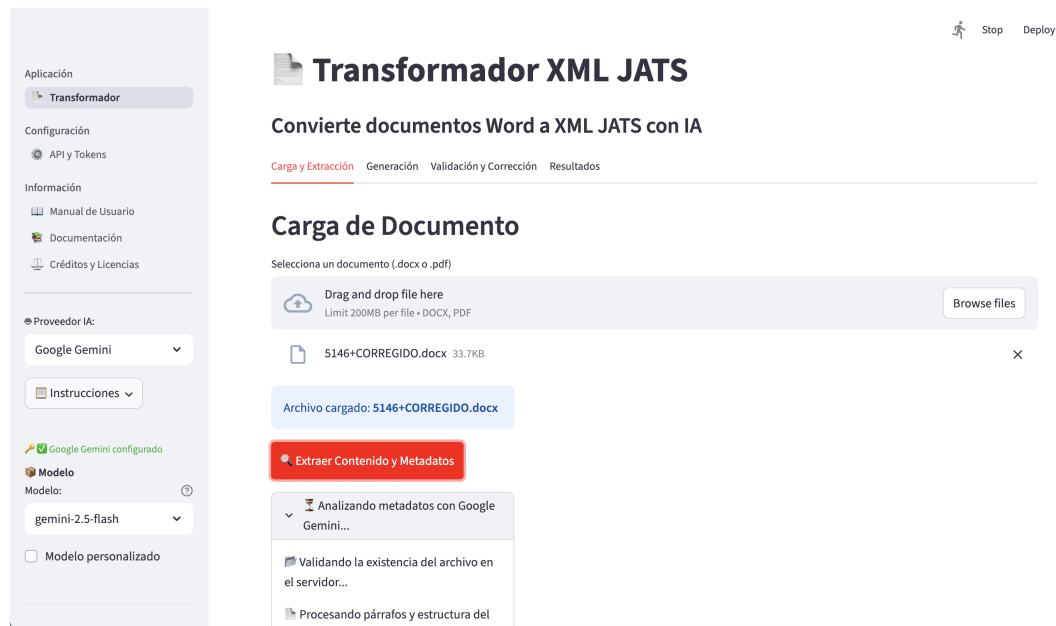


Figura 5: Área de carga de documentos. El sistema extrae automáticamente el contenido y los metadatos.

Paso 6 Revisar y Completar los Metadatos

Tras la extracción, el sistema muestra los metadatos detectados para que los revise y complete antes de generar el XML.

Metadatos que se extraen automáticamente:

Campo	Descripción
Título	Título completo del artículo
Autores	Nombre y apellido de cada autor
DOI	Identificador digital del artículo
Fecha de publicación	Año, mes y día de publicación
Fecha de recepción	Cuándo se recibió el manuscrito
Fecha de aceptación	Cuándo fue aceptado para publicación
Afiliaciones	Institución de cada autor
Resumen (Abstract)	Resumen del artículo
Palabras clave	Keywords del artículo

Qué hacer si faltan datos:

- Haz clic en cualquier campo editable y modifica el valor directamente.
- Si el DOI o la fecha de publicación no se detectaron automáticamente, el Chatbot Asistente se activará para pedírtelos. Puedes responder en el chat con el valor correspondiente.

- Revisa la sección de texto extraído para verificar que el contenido del artículo se capturó correctamente.
 - Revisa las tablas detectadas al pie de la sección para confirmar que las tablas se extrajeron con todos sus datos.
- > El DOI y la fecha de publicación son obligatorios para generar el XML JATS. Si no los tienes, el sistema no permitirá avanzar al siguiente paso.

Imagen correspondiente al Paso 6

Carga de Documento

Selecciona un documento (.docx o .pdf)

Drag and drop file here
Limit 200MB per file • DOCX, PDF

Browse files

5146+CORREGIDO.docx 33.7KB

Archivo cargado: 5146+CORREGIDO.docx

✓ Contenido y metadatos extraídos.

Re-extraer (Borrará cambios actuales)

> Vista Previa del Contenido (Texto)

Formulario de Metadatos Extraídos

Revisa los metadatos extraídos. Puedes editarlos antes de ir al Paso 2.

Título Artículo	Revista
PROMOCIÓN DEL BIENESTAR LABORAL Y	
Fecha (YYYY-MM-DD)	DOI
	10.22370/revmat.1.2026.5146
Autores (separados por coma)	
Franshesca Quijada Urzúa, Jovita Ortiz Contreras	

Guardar e ir al paso 2

Figura 6: Formulario de metadatos extraídos automáticamente. Todos los campos son editables antes de continuar.

Paso 7 Usar el Chatbot Asistente

Si hay información faltante o quieres ajustar los metadatos, el Chatbot Asistente te guía de forma conversacional.

Cuándo se activa el Chatbot:

- Automáticamente cuando el DOI o la fecha de publicación no se detectaron.
- Cuando quieres modificar o añadir información que el sistema no encontró.

Cómo interactuar con el Chatbot:

1. El chatbot aparece debajo del formulario de metadatos si detecta información faltante.
2. Lee el mensaje del chatbot te preguntará específicamente qué información necesita.
3. Escribe tu respuesta en el campo de texto (ej. 10.1234/revista.2024.001 para el DOI).

4. Presiona Enter o el botón de enviar.
5. El chatbot confirmará que recibió la información y actualizará los metadatos automáticamente.
6. Puedes continuar el diálogo si hay más datos incompletos.

Ejemplos de interacción:

Chatbot: No encontré el DOI del artículo. ¿Puedes proporcionarlo?

Tú: El DOI es 10.1234/rev.uv.2024.001

Chatbot: DOI registrado: 10.1234/rev.uv.2024.001

Chatbot: ¿Cuál es la fecha de publicación? (formato: día/mes/año)

Tú: 15 de marzo de 2024

Chatbot: Fecha de publicación: 15/03/2024

> También puedes usar el chatbot para pedir al asistente que reformule el resumen, corrija un nombre de autor o añada afiliaciones faltantes.

Paso 8 Seleccionar Modelos y Generar el XML JATS

Con los metadatos confirmados, es momento de generar el XML JATS. Puedes usar uno o múltiples modelos de IA simultáneamente.

Cómo generar el XML:

1. En la pestaña "Generación" del Transformador, verás el selector de modelos.
2. Selecciona uno o más modelos de IA de la lista desplegable:
 - Modo simple: selecciona un solo modelo para una generación rápida.
 - Modo Multi-Agente: selecciona 2-3 modelos para generar múltiples versiones en paralelo y elegir la mejor.
3. Presiona el botón " Generar XML JATS".
4. El sistema procesará el artículo aplicando el estándar JATS 1.4:
 - Etiquetará todas las secciones (<sec>, <title>, <p>).
 - Convertirá las tablas a <table-wrap> con <thead>/<tbody>.
 - Estructurará los metadatos en <article-meta>.
 - Formateará las referencias bibliográficas en <ref-list>.
5. Una barra de progreso mostrará el avance en tiempo real.

> La generación típicamente toma 30-90 segundos por modelo. Con múltiples modelos en paralelo, el tiempo total es similar al del modelo más lento.

> El sistema soporta artículos de hasta 65 536 tokens de salida equivalente a artículos de ~50 páginas sin truncamiento.

Imagen correspondiente al Paso 8



Figura 8: Interfaz de generación XML JATS. Selecciona los modelos de IA que competirán en la generación simultánea.

Paso 9 Leaderboard: Seleccionar el Mejor XML

Cuando generas XML con múltiples modelos, el Leaderboard te ayuda a comparar y seleccionar el resultado de mayor calidad.

Cómo funciona el Leaderboard:

1. Tras la generación, aparece un panel con una tabla comparativa de todos los modelos utilizados.
2. Cada modelo recibe:
 - Puntaje DTD (%): porcentaje de precisión estructural contra el DTD JATS oficial. Un 100% significa que el XML es perfectamente válido.
 - Auditoría Semántica: indicador de si el XML contiene placeholders (<!-- ... -->) o secciones vacías.
 - Densidad de texto: métrica que compara la cantidad de texto en el XML vs. el documento original.
3. El modelo con mayor puntaje es marcado automáticamente como "Ganador".
4. Puedes revisar el XML de cualquier modelo haciendo clic en "Ver XML" junto a su nombre.
5. Si prefieres usar un modelo diferente al ganador automático, selecciónalo manualmente.
6. Confirma tu selección con "Usar este XML" para proceder a la validación.

¿Qué hace que un XML sea de mejor calidad?

- Mayor cobertura del DTD (menos errores estructurales).
- Sin placeholders ni comentarios de IA en el contenido.
- Texto del artículo completamente transcrito (no resumido).
- Tablas correctamente estructuradas con todos los datos.

- Referencias bibliográficas completas y bien formateadas.

Imagen correspondiente al Paso 9

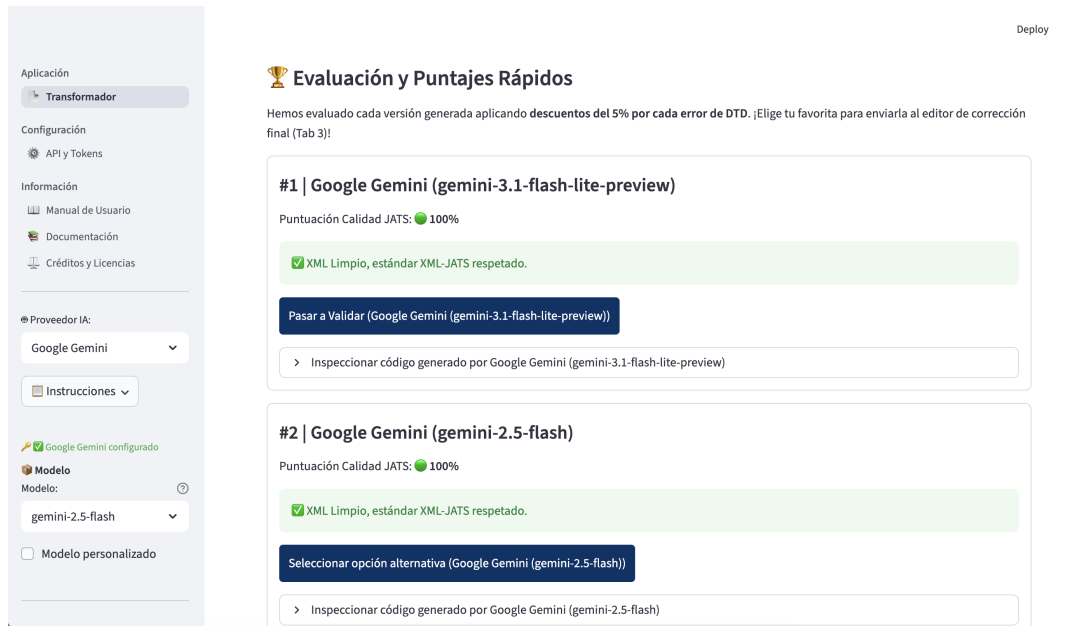


Figura 9: Leaderboard Multi-Agente con puntajes de calidad JATS. El ganador se determina automáticamente.

Paso 10 Validar el XML contra el DTD JATS

La validación DTD garantiza que el XML generado cumple con el estándar oficial JATS y puede ser procesado por sistemas de publicación como OJS.

Cómo ejecutar la validación:

1. En la pestaña "Validación", presiona el botón "Ejecutar Validación DTD".
2. El sistema valida el XML contra el DTD JATS 1.3 o 1.4 (según tu configuración) usando el DTD oficial empaquetado localmente sin necesidad de conexión a internet para esta etapa.
3. Los resultados pueden ser:
 - Validación exitosa: el XML cumple con el estándar JATS al 100%. Puedes proceder directamente a la descarga.
 - Errores encontrados: el sistema lista cada error con su línea y descripción. Se habilita el botón de corrección automática.

Errores comunes y sus causas:

Error	Causa probable
Element 'xxx' not expected	Etiqueta JATS no válida o mal anidada
Attribute 'xxx' not allowed	Atributo no soportado en esa versión JATS
Content model error	Orden incorrecto de elementos hijos
ID 'xxx' is not unique	Dos elementos con el mismo atributo id
IDREF error	Una referencia apunta a un ID que no existe

> No todos los errores impiden la publicación. Algunos son advertencias menores que el Agente Editorial puede corregir automáticamente.

Imagen correspondiente al Paso 10

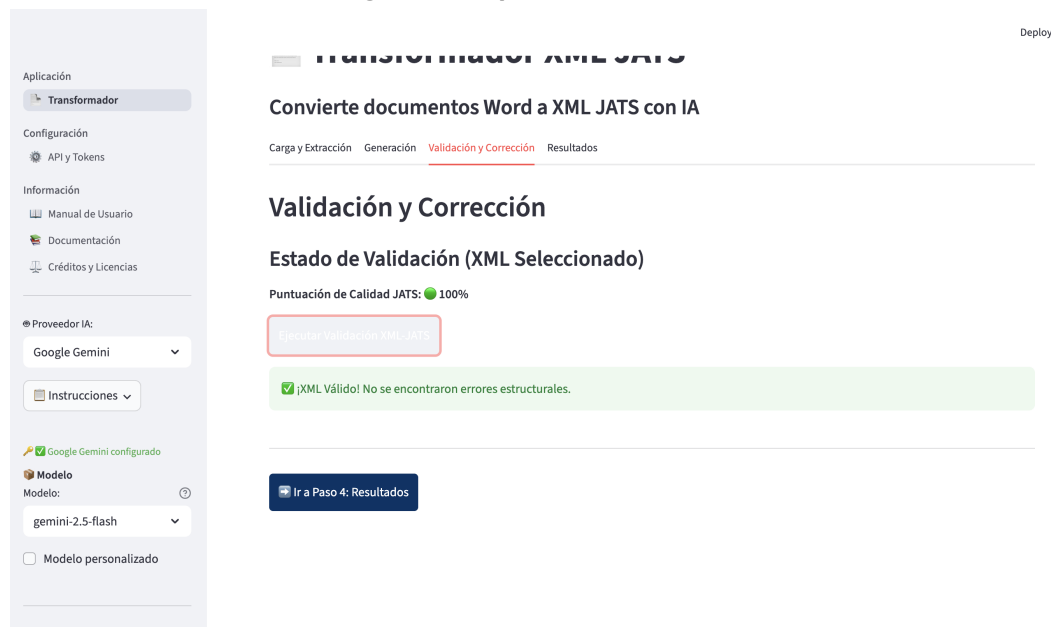


Figura 10: Resultado de la validación DTD JATS. Un XML válido al 100% puede descargarse y publicarse directamente.

Paso 11 Corregir Errores con el Agente Editorial

Si la validación encuentra errores, el Agente Editorial IA puede analizarlos y proponer correcciones automáticamente.

Cómo usar el Agente Editorial:

1. Tras una validación fallida, haz clic en "Intentar Solucionar con IA".
2. El sistema envía el XML con errores y el informe de errores DTD al Agente Editorial.
3. El Agente analiza cada error y genera una versión corregida del XML, preservando completamente el texto original del artículo solo modifica la estructura de etiquetas.
4. Una vez generada la corrección, el sistema ejecuta automáticamente una segunda validación.
5. Si quedan errores menores, puedes repetir el proceso (se recomiendan máximo 3 iteraciones).

Corrección manual alternativa:

Si prefieres corregir el XML manualmente:

1. Haz clic en " Editar XML" para abrir el editor de texto integrado.
2. Modifica las líneas con error según el informe de validación.
3. Presiona " Aplicar cambios" para guardar y volver a validar.

El Chatbot de Corrección:

También puedes dialogar directamente con la IA para correcciones específicas:

Tú: La tabla de resultados no tiene thead, solo tbody. ¿Puedes añadirlo?

Agente: He añadido el elemento <thead> con los encabezados detectados...

> Si después de 3 iteraciones de corrección automática persisten errores, revisa el documento Word original puede tener estructuras inusuales que requieren edición manual del XML.

Paso 12 Vista Previa HTML y Exportación

Una vez validado el XML, puedes generar una vista previa HTML y descargar los archivos finales.

Generar la Vista Previa HTML:

1. En la sección "Resultados", presiona " Generar HTML".
2. El sistema convierte el XML JATS a un archivo HTML5 responsivo con:
 - Logo de la universidad incrustado (Base64).
 - Tabla de contenidos interactiva con anclas a cada sección.
 - Referencias bibliográficas con DOIs clickeables que enlazan a doi.org.
 - Soporte para modo claro y oscuro.
 - Diseño responsivo para móviles y escritorio.
3. Haz clic en " Ver vista previa en nueva pestaña" para abrir el HTML en el navegador.
 - Los enlaces internos (citas [1], [2]) navegan correctamente dentro del artículo.
 - Los DOIs en las referencias son clickeables y abren doi.org.
4. Revisa el HTML y confirma que el artículo se ve correctamente antes de descargarlo.

Descargar los archivos:

Presiona los botones de descarga para obtener:

- Descargar XML JATS: el archivo .xml validado, listo para subir a OJS u otro sistema editorial.
- Descargar HTML: la vista web del artículo para publicación o archivo.
- Descargar Manual PDF: el manual de usuario completo en formato PDF.

> El archivo XML descargado incluye la declaración <!DOCTYPE> correcta para la versión JATS seleccionada (1.3 o 1.4), lista para ser importada directamente en Open Journal Systems (OJS).

Imagen correspondiente al Paso 12

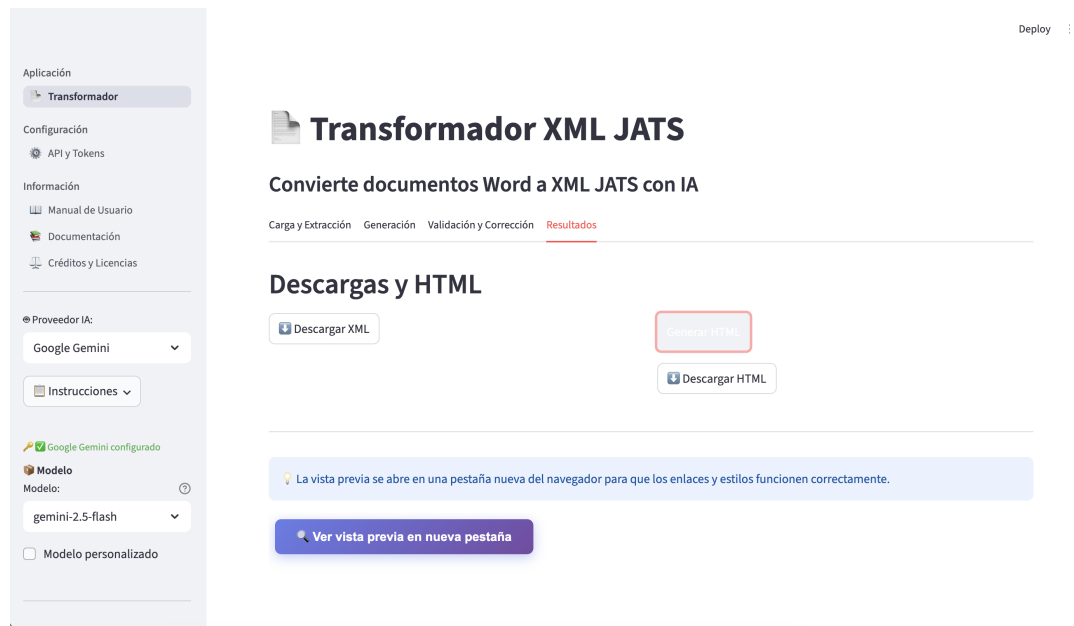


Figura 12: Vista previa HTML del artículo con tabla de contenidos interactiva y referencias con DOIs clickeables.

Preguntas Frecuentes

¿Qué formatos de documento soporta la herramienta?

Soporta archivos Microsoft Word (.docx) y documentos PDF (.pdf). El formato .docx es preferido ya que permite una extracción más estructurada del contenido, tablas e imágenes. Los PDFs con texto seleccionable funcionan bien; los PDFs escaneados (imágenes) tienen extracción limitada.

¿Necesito conocimientos de XML o JATS para usar la herramienta?

No. La herramienta está diseñada para usuarios editoriales sin conocimientos técnicos en XML. La IA se encarga de todo el etiquetado. Sin embargo, si deseas editar el XML manualmente, el editor integrado y el chatbot pueden guiarte.

¿Qué hago si falla la validación DTD?

Usa el botón "Intentar Solucionar con IA" para que el Agente Editorial corrija los errores automáticamente. Si persisten tras 3 intentos, revisa que el documento Word no tenga tablas anidadas complejas ni cuadros de texto flotantes. También puedes editar el XML manualmente usando el editor integrado.

¿Las tablas del documento se incluyen en el XML?

Sí. Las tablas se extraen automáticamente y se convierten a formato JATS (<table-wrap>) con encabezados (<thead>) y cuerpo (<tbody>) correctamente estructurados, preservando todas las filas y columnas del original. Las tablas muy complejas con celdas combinadas pueden requerir revisión manual.

¿Se incluye el DOI y la fecha de publicación en el XML?

Sí. El DOI y la fecha de publicación son campos obligatorios que se extraen automáticamente. Si el sistema no los detecta en el documento, el Chatbot Asistente te los pedirá antes de permitir la generación del XML.

¿Los caracteres especiales (tildes, ñ, etc.) se preservan correctamente?

Sí. El sistema usa codificación UTF-8 en todo el flujo (extracción, generación XML y conversión HTML). Los caracteres acentuados, la ñ, el ü y otros caracteres especiales del español se preservan correctamente.

¿Los enlaces del HTML funcionan correctamente?

Sí. Los enlaces de citas (ej. [1], [2]) y la tabla de contenidos navegan correctamente dentro de la página HTML. Los DOIs en las referencias bibliográficas son clickeables y enlazan directamente a doi.org. Usa siempre el botón "Ver vista previa en nueva pestaña" para la mejor experiencia de navegación.

¿Es seguro subir mis archivos?

Los archivos se procesan temporalmente en memoria del servidor y no se almacenan permanentemente. El texto del artículo se envía a la API del proveedor de IA seleccionado (ej. Google Gemini) bajo los términos de servicio de ese proveedor. Para máxima privacidad, usa Ollama (inferencia local) el contenido no sale del servidor.

¿Mi API Key está segura?

Sí. La clave se almacena localmente en data/config.db (carpeta excluida de Git) con ofuscación Base64. Una vez guardada, desaparece de la interfaz y solo se muestra enmascarada (ej. AlzaxY4Z). La clave nunca se envía a ningún servidor externo distinto al proveedor de IA correspondiente.

¿Qué pasa si alcanzo el límite de tokens gratuitos de Gemini?

El panel de consumo en "API y Tokens" muestra tu consumo actual vs. los límites del tier gratuito. Al alcanzar el 80%% se muestra una advertencia; al 100%% los requests fallarán con error 429. Puedes:

- Esperar a las 00:00 UTC para que el límite diario se renueve.
- Cambiar a gemini-2.0-flash (1500 req/día gratuitas).
- Activar facturación en Google Cloud para uso Pay-as-you-go con la misma clave.
- Configurar otro proveedor (OpenAI, Groq, etc.) como alternativa.

¿Puedo usar el sistema sin conexión a internet?

Parcialmente. Si usas Ollama o LM Studio (proveedores locales), la generación de XML funciona sin internet. Sin embargo, la validación DTD usa el DTD empaquetado localmente, por lo que también funciona offline. Solo la interfaz web y la descarga del manual PDF requieren conexión.

¿Qué versión de JATS debo usar: 1.3 o 1.4?

Usa JATS 1.4 a menos que tu sistema editorial (OJS, etc.) especifique explícitamente que requiere 1.3. JATS 1.4 añade mejoras en accesibilidad, matemáticas estructuradas y afiliaciones. La versión se configura en "API y Tokens" "Estándar XML JATS".

¿Puedo procesar múltiples artículos a la vez?

La interfaz web procesa un artículo a la vez. Para procesamiento por lotes, usa la interfaz de línea de comandos (CLI):

```
bash
```

```
python -m modules.transformer articulo1.docx salida1.xml
```

```
python -m modules.transformer articulo2.docx salida2.xml
```

Créditos y Licencias

Esta herramienta ha sido desarrollada para optimizar el flujo de trabajo editorial de las Revistas UV, automatizando la conversión de manuscritos a XML JATS validado.

Desarrollador Principal:

Cristian Carreño León (cristian.carreno@uv.cl)

Escuela de Obstetricia y Puericultura

Facultad de Medicina

Universidad de Valparaíso, Chile

Tecnologías Utilizadas:

- Python 3.9+: Lenguaje base.
- Streamlit 1.51+: Framework de interfaz de usuario web.
- Google Gemini 2.5 Flash: Modelo de lenguaje (LLM) principal para etiquetado inteligente.
- OpenAI / Anthropic / DeepSeek / Mistral / Groq: Proveedores alternativos de IA.
- Ollama / LM Studio: Inferencia local sin costo.
- lxml 6.0+: Procesamiento, validación y parsing de XML/HTML.
- python-docx 1.2+: Extracción de contenido desde archivos Word.
- pdfplumber 0.10+: Extracción de texto desde PDFs.
- tenacity 9.1+: Reintentos robustos con backoff exponencial.
- FPDF2: Generación de manuales en PDF.
- SQLite: Persistencia de API keys y métricas.
- JATS 1.3 / 1.4 (ANSI/NISO Z39.96-2024): Estándar de etiquetado XML.

Privacidad y Tratamiento de Datos:

1. Procesamiento Volátil: Los archivos no se almacenan permanentemente en el servidor.
2. API de Inteligencia Artificial: El texto enviado a la API se procesa bajo las políticas del proveedor seleccionado.
3. Inferencia Local: Con Ollama/LM Studio, ningún contenido sale del servidor (privacidad total).
4. Claves API: Almacenadas localmente en data/config.db con ofuscación, excluidas de Git.

Licencia:

Software de uso exclusivo para la Universidad de Valparaíso. Todos los derechos reservados.

El código y la documentación son propiedad de la Universidad de Valparaíso y su autor.

Version: 0.7.5